CALIFORNIA POLYTECHNIC STATE UNIVERSITY

Final Project: Fully Connected Spiking Neural Network with Address-Event Representation

Presented to:

Dr. Chakraborty

Presented by:

Marcus Ivey

Sanjeev Srinivasan

Isaac Thomas

March 13, 2026

Table of Contents

Table of Contents **2**
Introduction/Background **4**
Motivation 4
Theory of Operation 4
Methodology **5**
Modules 5
Input Encoder 5
Fig. 1: Delta Modulator Block Diagram 5
LIF Neurons 6
Fig. 2: LIF Difference Equation Hardware Optimization 6
Fig. 3: LIF Neuron DSP Diagram 7
AER Broadcast Handler 7
Fig. 4: Example of a Dense Direct Connection Topology 8
Fig. 5: AER Broadcast Bus 8
Synapse Accumulator 9
Fig. 6: Synapse Accumulator DSP Diagram 9
Output Decoder 10
Fig. 7: Output Decoder Signal Flow 10
End-to-End Top Module 11
Fig. 8: Top-Level FSM Diagram 11
Fig. 9: End-to-End Top-Level Module 12
Verification Strategy 13
RTL to GDSII Flow 13
Results **14**
Verification 14
Fig. 10: ECG Input Signal, Spike Raster, and Output Classification Signals 14
Synthesis & Layout 14
Table I: ASIC Implementation Metrics 15
Fig. 11: ASIC Implementation Layout 15
Analysis **16**
Verification 16
Synthesis & Layout 16
Conclusion **17**
Appendix **18**
Verification Unit Test Screenshots 18

Fig. 12: Input Encoder Verification 18
Fig. 13: Neuron Unit Test Verification 18
Fig. 14: AER Handler Unit Test Verification 19
Fig. 15: Synapse Accumulator Unit Test Verification 19
Fig. 16: Output Decoder Unit Test Verification 20
Fig. 17: Top Module Verification 20
Fig. 18: Top Module Verification 20
Config.json 21
Modules, Testbenches, Netlist, etc. 22

Introduction/Background

Motivation

Machine learning hardware acceleration is a growing field that aims to improve the speed and power efficiency of model training and inference. One particular type of machine learning paradigm, Spiking Neural Networks (SNNs), promises to significantly reduce the challenges of hardware implementation by simplifying routing between neurons. SNNs achieve this by taking inspiration from biological neurons, which transmit signals as all-or-nothing “action potentials” or spikes that can be represented by a single bit. This transforms the data representation and computation from multiple expensive matrix multiplications to a dynamic, sparse, and time-dependent representation and computation.

The main inspiration for this project is that all three team members are currently working on spiking neural network projects. This allowed each of us to apply our domain-specific knowledge to our respective parts of the project and achieve a more ambitious goal.

Theory of Operation

Our SNN is designed for extremely fast and efficient inference on real-time ECG data. The parameters and weights of each connection are constant at synthesis time, making the system very application-specific. These baked-in weights allow the design to operate extremely efficiently across all metrics.

The SNN accepts a continuous-time input signal, such as an ECG signal, and classifies it into one of two classes (binary classification). The signal goes through an encoding stage to convert it into spikes, 4 hidden SNN layers of processing, and an output decoder that provides the binary classification. One of the primary features that enables power and area efficiency is the AER broadcast handler, which time-multiplexes spike propagation between layers over a single bus. Since spike events are sparse, the timing cost is minimal, but the gains on routing resources, power efficiency, and scalability cannot be overstated.

Methodology

Modules

The system was designed hierarchically, starting with individual submodules and clearly defined I/O contracts, and then integrating them into the top-level module.

Input Encoder

To convert a sampled continuous-time biomedical signal into a spike stream, we used delta modulation as our spike-encoding scheme. Delta modulation generates a binary spike when the difference between consecutive samples exceeds a configurable threshold. Specifically, it generates two spike streams: an ‘UP’ and ‘DOWN’ spike stream, depending on whether the difference is greater or less than the threshold, respectively. The threshold can be changed to achieve the desired resolution and spike density. Fig. 1 shows the resulting spike streams from applying delta modulation to a raw ECG input stream.

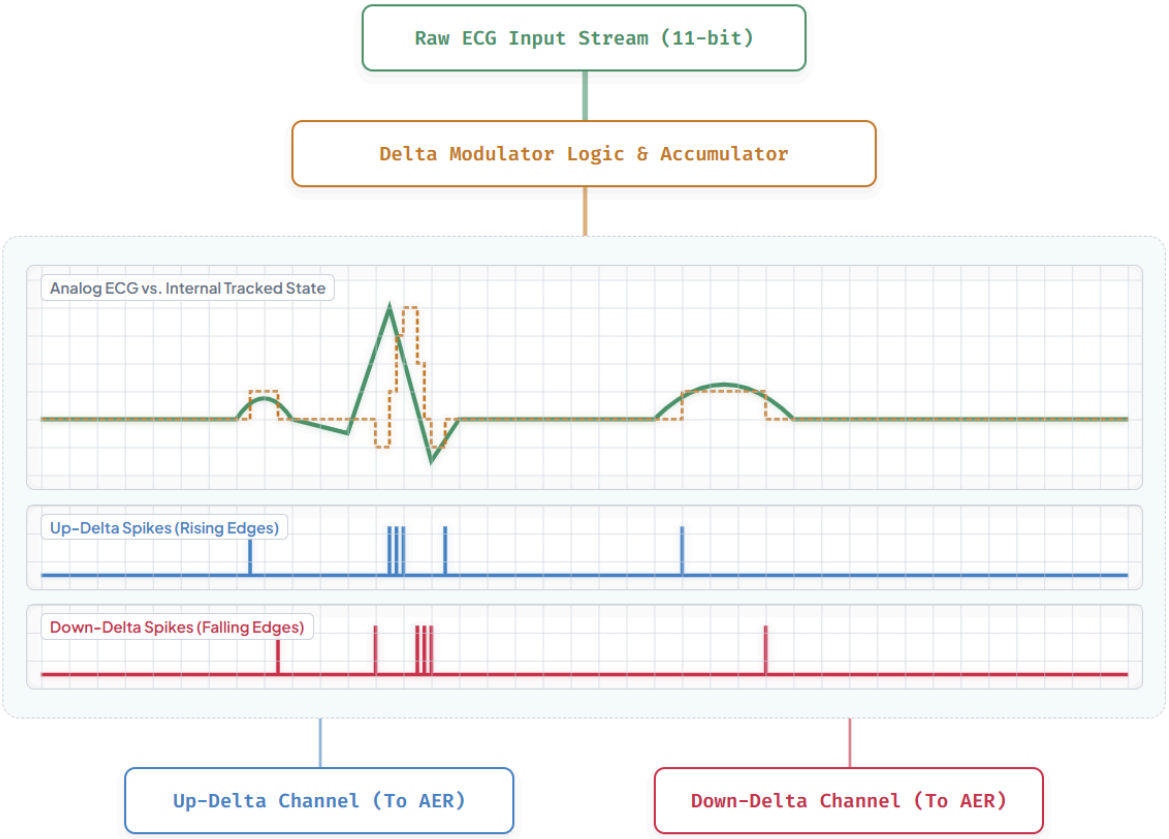

Fig. 1: Delta Modulator Block Diagram

LIF Neurons

The Leaky Integrate-and-Fire (LIF) neuron model is perhaps the most widely used in software and hardware SNNs due to its extreme efficiency. We took the typical difference equation used by computational neuroscientists and aggressively optimized it for hardware, entirely eliminating the need for hardware multipliers and utilizing the full register state space. This optimization was achieved by adjusting the constants in the difference equation to simplify and eliminate entire arithmetic steps while preserving the model's dynamics.

TYPICAL DIFFERENCE EQUATION

$$v[n+1] = A \cdot v[n] + B \cdot I[n] + C$$

↓ (A=510/512, B=1, C=0) ↓

HARDWARE OPTIMIZED

$$v[n+1] = v[n] \cdot 0.996 + I[n]$$

↓ (Multiplier-less RTL) ↓

BIT-SHIFT IMPLEMENTATION

$$v[n+1] = ((v[n] \ll 9) - (v[n] \ll 1)) \gg 9 + I[n]$$

Fig. 2: LIF Difference Equation Hardware Optimization

The choice to use 0.996 allows us to apply the 510/512 approximation, implemented with shifts and adds, as shown in the equation above. We optimized the constant B to 1 because the input current weights can simply be scaled to account for this. By setting the constant C to 0, the resting potential that the neuron decays to becomes 0, which, for our 16-bit signed state space, is right in the middle of the range. This allows for the maximum amount of positive and negative headroom.

Not shown in the above equations are the threshold and reset voltages. The threshold voltage was set to the maximum possible state value, enabling full-range utilization of the state registers and reuse of the saturation logic. The reset voltage was set at the midpoint of the negative range, thereby preserving the refractory-period dynamics while allowing sufficient negative headroom for strong inhibitory signals. Hardware saturation is enforced by allowing the intermediate wires to grow to their worst-case widths during combinational arithmetic, then clipping them to the min and max register values before storage.

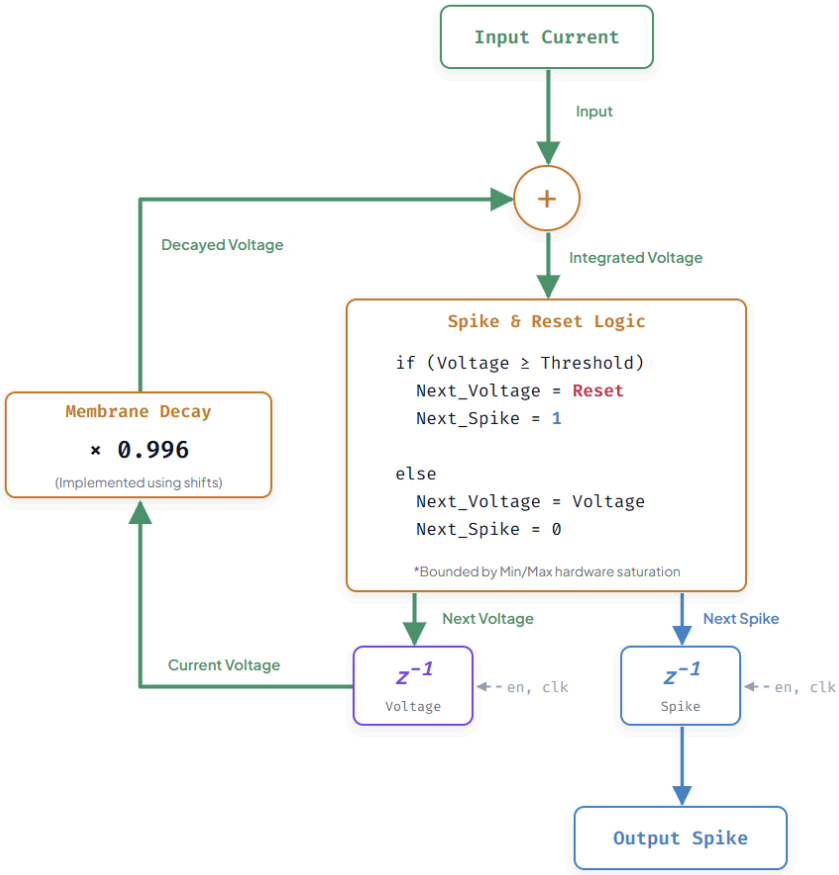

Fig. 3: LIF Neuron DSP Diagram

AER Broadcast Handler

Our design employs dense layer connections, which do not scale well. For our model, this was manageable since there are at most 64 single-bit connections between two 8-neuron layers, as shown in Figure 4. To improve scalability, we connected the layers via a broadcast bus that implements the Address Event Representation (AER) protocol, as shown in Figure 5.

The AER Handler module employs a static-priority centralized arbiter that searches for the first master (the spike output register of the previous layer) with a spike register of 1 that hasn't been serviced yet. During each clock cycle in the propagation phase, the arbiter selects the highest-priority spike address and broadcasts it on the bus. It maintains an internal flag register for each master, indicating if it has been serviced during that phase. At the end of the phase, the AER handler sets a flag indicating all masters have been serviced. When it receives the acknowledged handshake, it clears the serviced registers and waits for the next cycle. Note that the address bus is combinational rather than registered, allowing the Synapse Accumulator to decode in the same clock cycle.

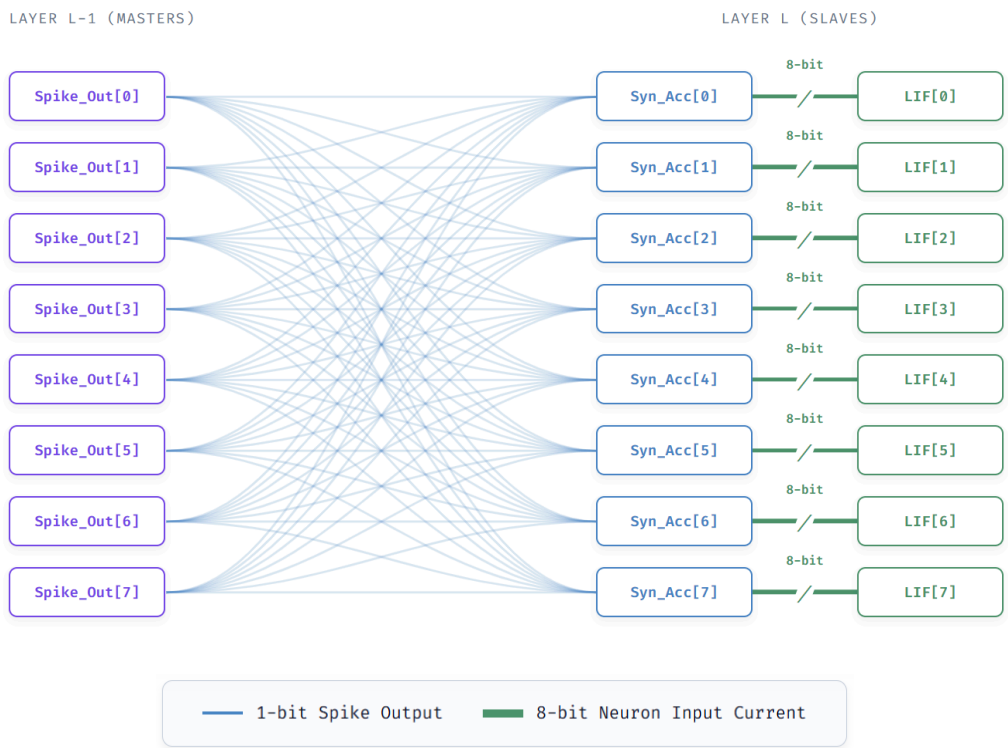

Fig. 4: Example of a Dense Direct Connection Topology

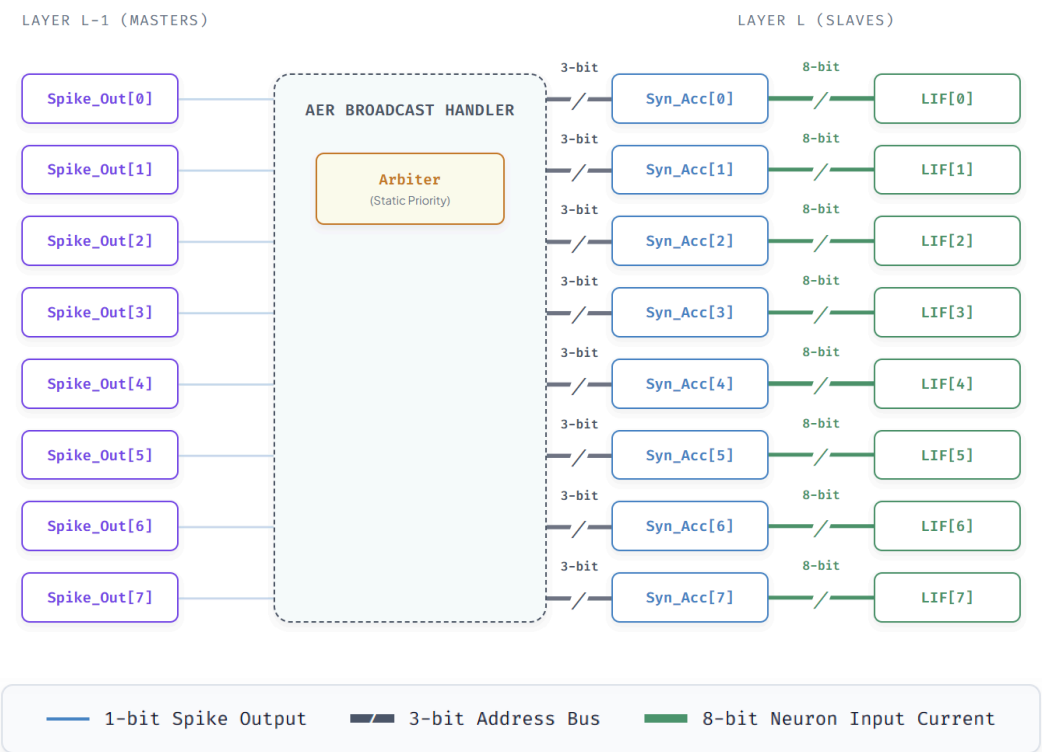

Fig. 5: AER Broadcast Bus

Synapse Accumulator

The synapse accumulator performs two functions: Decode addresses from the AER bus to weights, and add those weights to an accumulation register. This makes the Synapse Accumulator a decentralized decoder for the AER bus and, therefore, it operates during the same propagation phase. As mentioned previously, the weights are determined and fixed before synthesis, and are therefore baked into the decoder as a combinational lookup table. As with the neurons, the Synapse Accumulator enforces saturation logic before storing in the state register to avoid overflow or underflow.

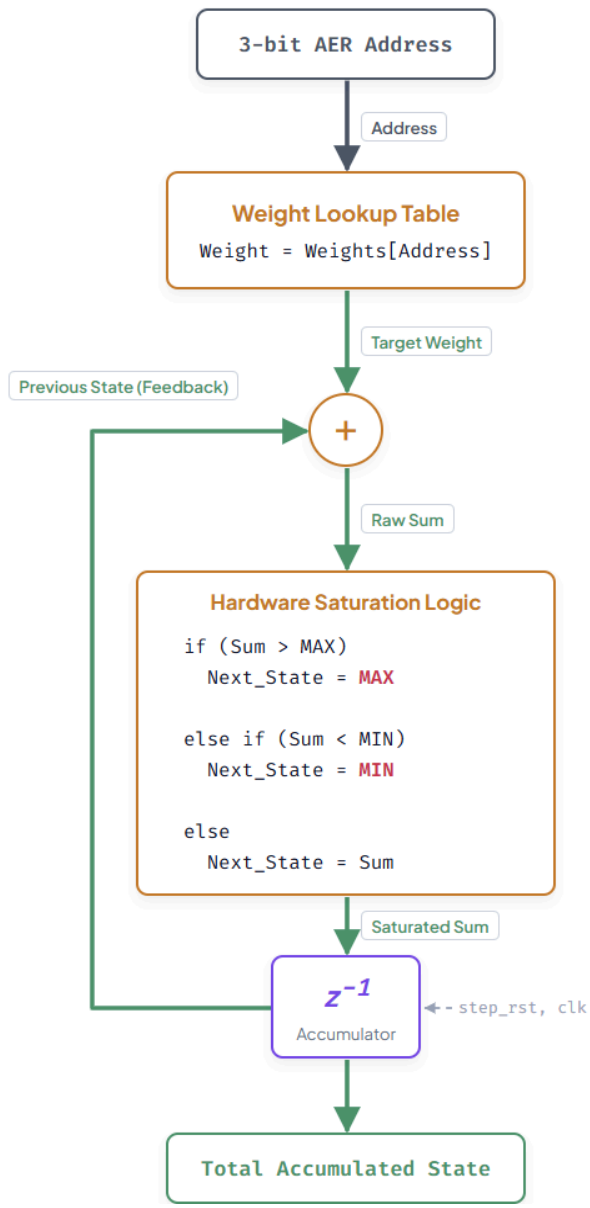

Fig. 6: Synapse Accumulator DSP Diagram

Output Decoder

In order to convert the final stage of neurons into a binary classification, a bank of low-pass filters is used as a temporal integrator, and a multiply-accumulate unit is used as a final classifier, followed by a comparator. The block diagram below shows the flow from final neuron spikes to binary classification.

The LPF unit implements a digital leaky integrator, similar to the leaking computation done in each neuron. It transforms the discrete spike event stream into a continuous activity level. A bank of these filters essentially performs a moving average of the neural activity for each neuron. Then, the top output decoder module snapshots the values of the LPF bank (on start) to capture one coherent view of the neuron activity. Then, the MAC unit uses the snapshot weight and value for each neuron to serially multiply and accumulate values into a classification. Finally, this value is evaluated against a threshold to produce a binary classification.

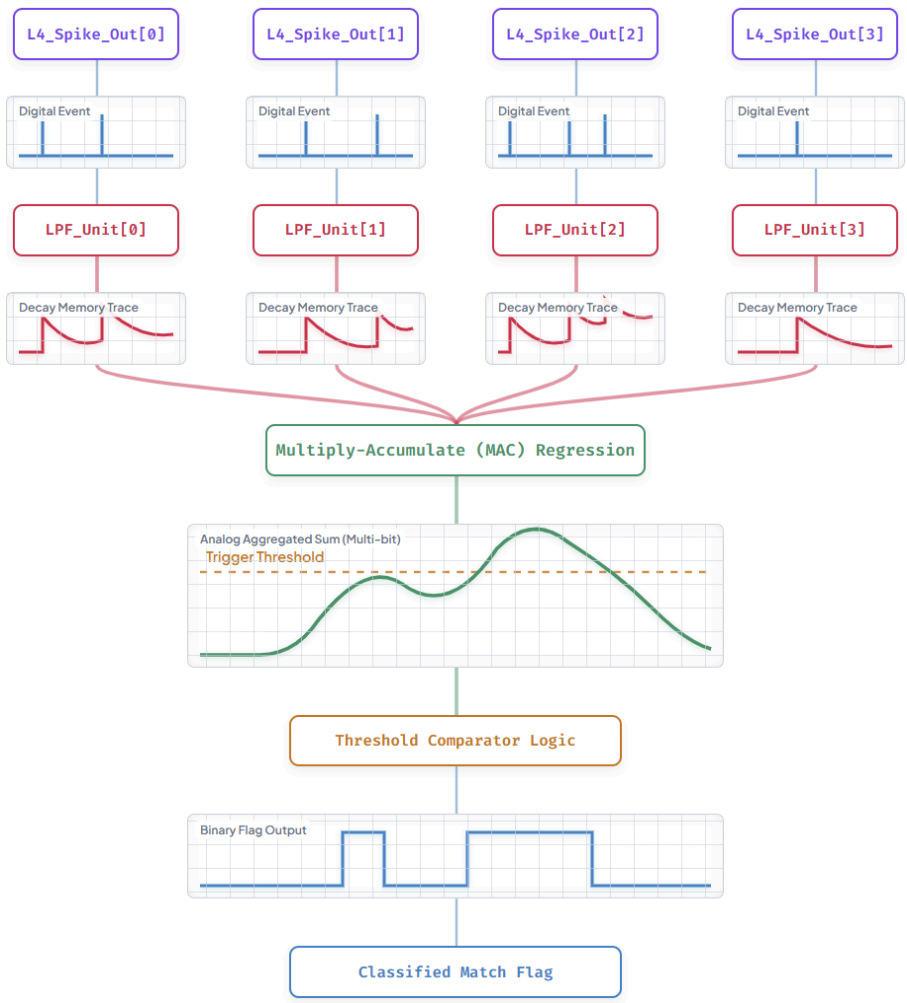

Fig. 7: Output Decoder Signal Flow

End-to-End Top Module

The top-level module implements all submodules, manages the data flow and timing between them, and uses a Finite State Machine (FSM) to handle handshaking and flagging of input and output signals, as well as pipeline stages. For each input, the layers are run for 10 cycles, which we found through heuristic testing to be sufficient to capture the timing dynamics. The design is pipelined, so each layer operates in parallel, meaning the system requires fill and drain times (only 5 cycles, which is negligible). The number and width of the hidden layers are parameterizable and use nested generate statements to accomplish this. For our final design, we went with 4 layers, 8 neurons wide.

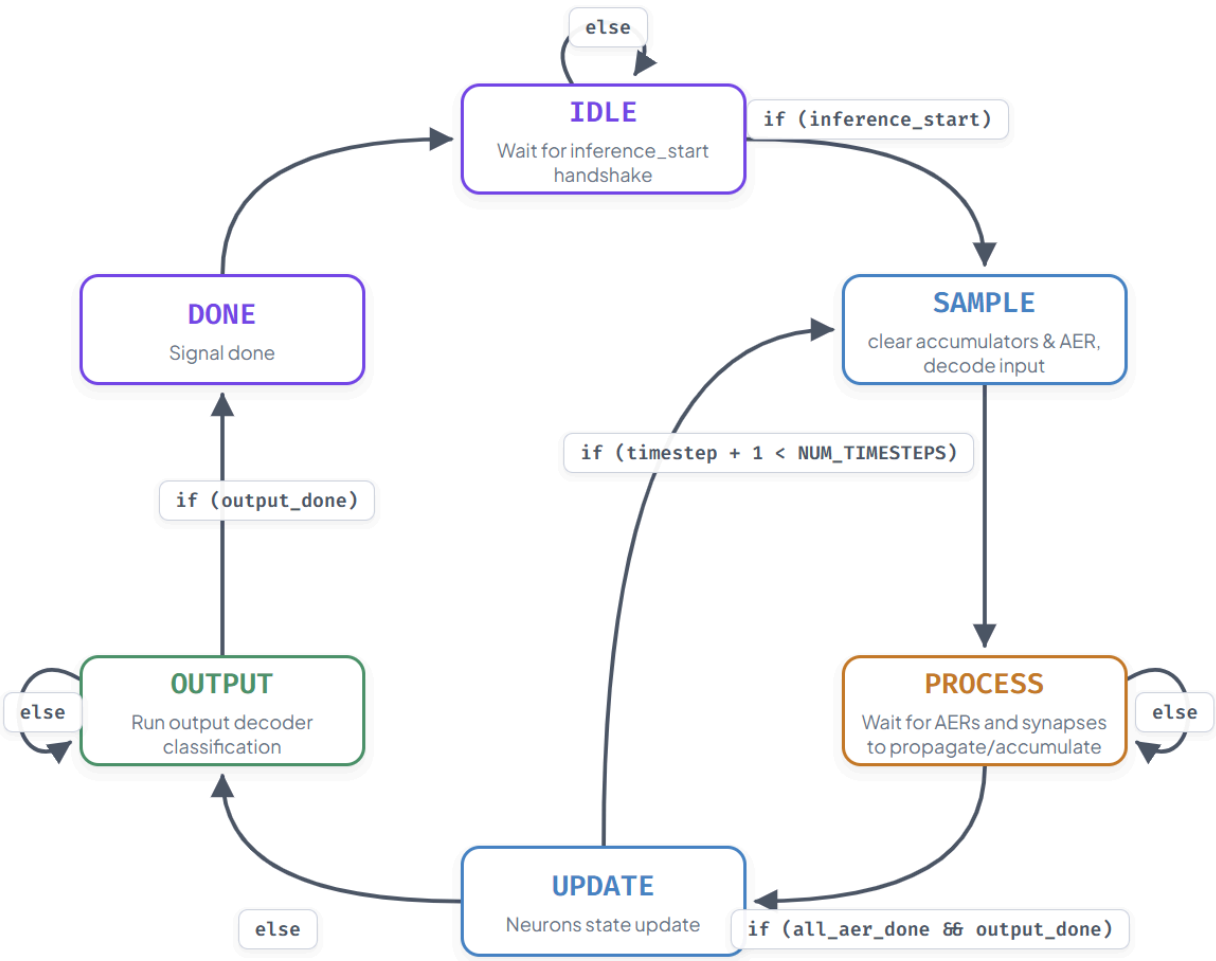

Fig. 8: Top-Level FSM Diagram

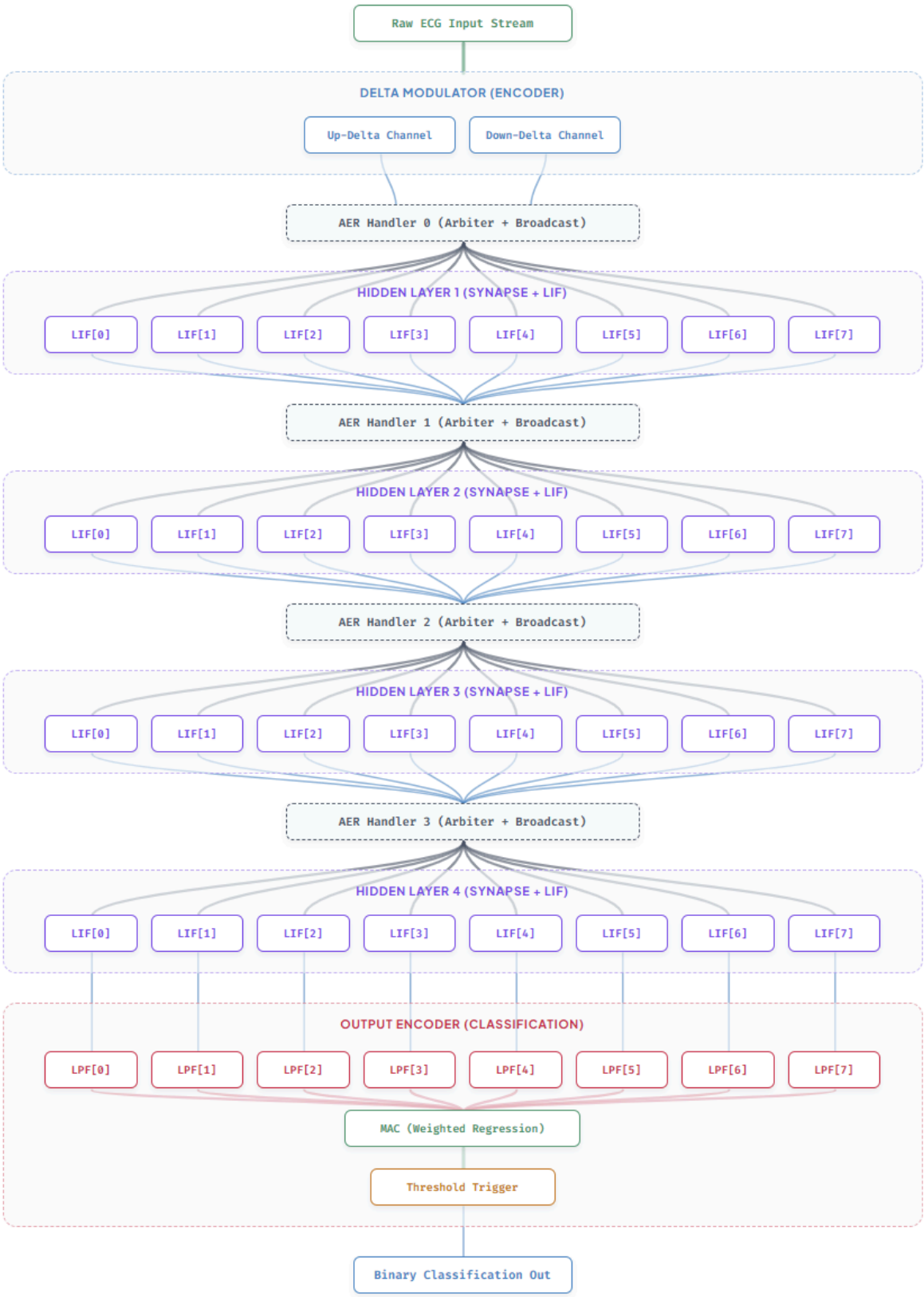

Fig. 9: End-to-End Top-Level Module

Verification Strategy

For the Neuron, Synapse Accumulator, Output Decoder, and AER Handler modules, we implemented testbenches that each include multiple unit tests that assert the expected outputs. These unit tests range from normal operation examples to corner cases and stress tests. These tests are run on a pass/fail basis, and the outcome of each test is logged to the terminal, including a tally of passes and failures.

For the Input Encoder and Top Modules, we implemented testbenches that apply a test ECG signal and monitor the output. The ECG signal is taken from the MIT-BIH Dataset; the continuous-time signal is sampled at 360Hz, with 11-bit resolution over a 10 mV range. For the Input encoder, we simply wanted to confirm the presence of spikes on the up and down channels. For the top module, we exposed and examined multiple internal signals to assess expected behavior, including FSM states, handshake and flag signals, spike rasters, and output classification. These signals were examined both in Vivado's waveform viewer and saved to a file for later analysis using Python plotting scripts.

RTL to GDSII Flow

To successfully complete the RTL-to-GDSII flow for our system, several changes were made to the SystemVerilog modules before attempting synthesis. The most significant change was flattening multi-dimensional arrays used for readability and simplicity in simulation into 1D vectors. This required complex, specific bit-slicing to effectively pass the same information between modules. Additionally, we removed any unintentional, inferred latches in our logic. Finally, to resolve errors during header parsing, the weights package was integrated directly into the top module.

The goal of our implementation was to provide 'loose' timing constraints (25 ns/40 MHz clock) and optimize the synthesis strategy for low area. The config.json file in the Appendix shows the constraints used for the Librelane flow. To manage congestion, core utilization was reduced to 40%.

Results

Verification

All unit tests passed for the modules that utilized them. The Input Decoder's up and down channels produced spikes as expected. See the appendix for screenshots of test bench outputs.

The top-level module's test bench produced the expected results and saved data to a .txt file, which we analyzed with Python.

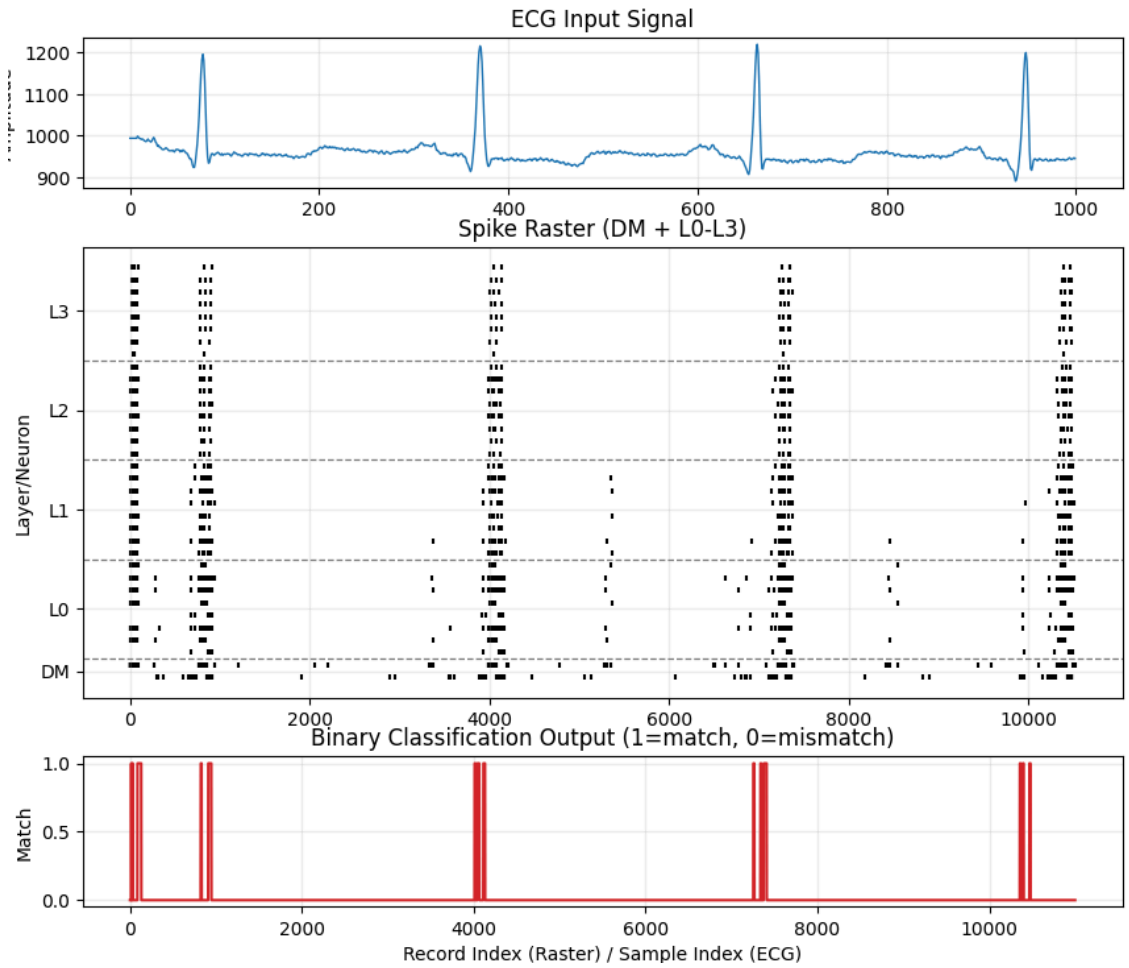

Fig. 10: ECG Input Signal, Spike Raster, and Output Classification Signals

Synthesis & Layout

The RTL-to-GDSII flow produced a DRC-free layout that passes LVS. Table I below presents key implementation metrics, and Figure 11 shows the resulting layout.

Table I: ASIC Implementation Metrics

Resource Metric	Value
Die Area (mm^2)	0.402
Core Area (mm^2)	0.379
Instance Area (mm^2)	0.150
Instance Count	20,823
Total Power (mW)	71.943
Switching Power (mW)	38.679
Worst Slack @ 40 MHz (ns)	4.09

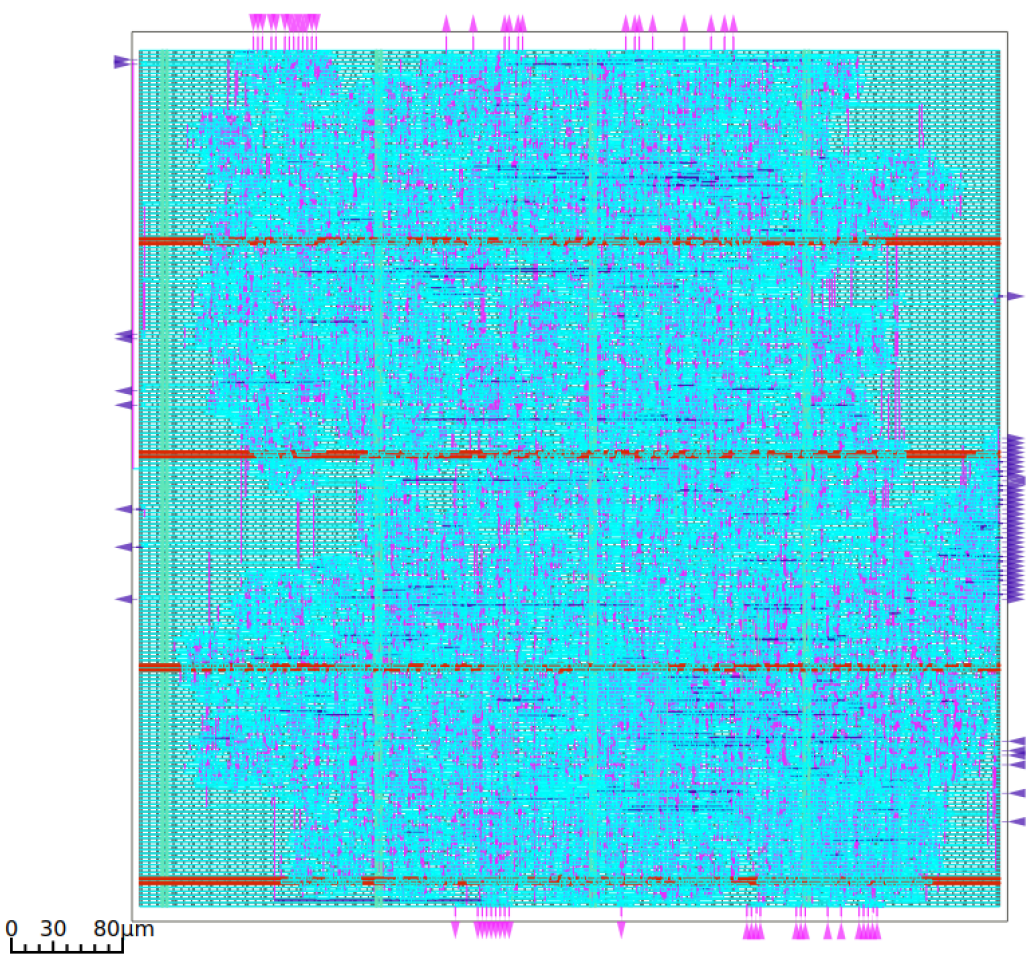

Fig. 11: ASIC Implementation Layout

Analysis

Verification

The Plot generated by the Python script in Figure 10 shows the input ECG signal, the spike raster of each spiking layer, and the output classification for the end-to-end system. The spike raster is divided into layers with the Delta Modulator up/down channels (DM) starting at the bottom and progressing upward through the layers. We can see spikes in each layer, which primarily coincide with the QRS complex of the ECG signal, as does the binary classification output in the bottom plot. Because the weights are randomly initialized and untrained, no real classification occurs, and the spike dynamics are unoptimized, but this still proves that the system works end-to-end.

It is interesting to note that there is some startup transience in the first few inferences, likely due to a mismatch between the reset value of the Delta Modulator's internal signal representation and the ECG input signal. It is also interesting that the top module test bench completed in 650 µs at 40 MHz. Because the ECG input we used is roughly 5 seconds long, if we operated this design in real time, we would be in the idle state about 99.9% of the time, which means we could easily implement clock gating during the idle state to save power.

Synthesis & Layout

The results from the 130nm are an area-efficient design, with the chip occupying less than 0.5 mm^2 of area. The logic density is concentrated at the center of the chip, with breathing room near the edges, supporting the use of 40% core utilization. The compact die area shows the validity of scaling this design. The design has a 16% timing margin for the 25ns clock. Furthermore, more than half of the power comes from switching; the event-driven nature of an SNN system results in sparse spikes. This suggests that a majority of the dynamic power consumption is consumed by the clock tree/global clock distribution. This promotes the future implementation of clock gating to significantly reduce the switching power consumption. Finally, the design changes made before performing synthesis proved to be functional, as the final layout is devoid of errors.

Conclusion

This project successfully demonstrated the hardware implementation of a highly efficient Spiking Neural Network designed for real-time ECG classification. By utilizing an Address-Event Representation (AER) broadcast bus instead of dense layer connections, alongside a heavily optimized, multiplier-less Leaky Integrate-and-Fire (LIF) neuron model, the architecture achieved significant scalability and efficiency. End-to-end verification confirmed the system's functional data path and spiking dynamics. Furthermore, the RTL-to-GDSII flow produced a DRC-free layout occupying only 0.402 mm^2 of die area.

Ultimately, the event-driven nature of the SNN results in sparse spikes and substantial idle time, highlighting clock gating as a highly viable future optimization to further reduce dynamic power consumption.

Appendix

Verification Unit Test Screenshots

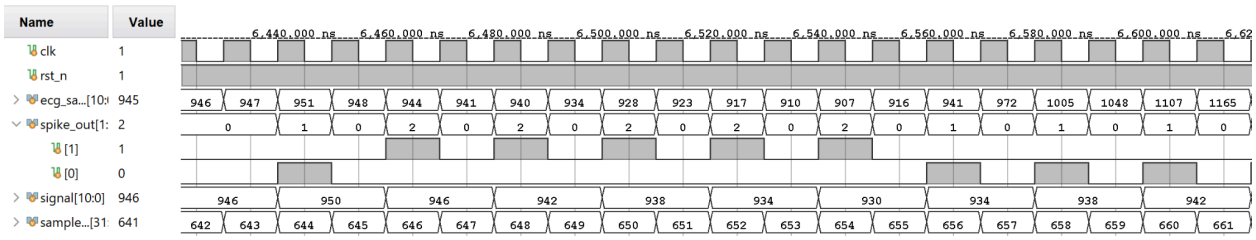

Fig. 12: Input Encoder Verification

```
run all
[PASS] T1 Reset spike=0
[PASS] T2 Enable gating - spike still 0
[PASS] T3 Decay - |v| decreased
[PASS] T4 Positive input causes spike
[PASS] T4 V_RESET after spike
[PASS] T5 Post-spike recovery - spikes again
[PASS] T6 Negative saturation - v_reg = MIN_VAL
[PASS] T6 Negative saturation - no spike
[PASS] T7 Zero input, zero state - v_reg = 0
[PASS] T7 Zero input - no spike
[PASS] T8 Input alignment - shifted = 16 * default

==========================
  LIF NEURON FIXED TB
  PASS: 11  FAIL: 0
==========================
$finish called at time : 1030 ns : File "Z:/Coursework/EE531/Final/tb_lif_neuron_fixed.sv" Line 225
```

Fig. 13: Neuron Unit Test Verification

```
run all
[PASS] T1 Reset - aer_valid=0
[PASS] T2 Single spike - valid
[PASS] T2 Single spike - addr
[PASS] T2 After servicing - valid=0
[PASS] T3 Priority - first grants neuron 1 - valid
[PASS] T3 Priority - first grants neuron 1 - addr
[PASS] T3 Priority - next grants neuron 3 - valid
[PASS] T3 Priority - next grants neuron 3 - addr
[PASS] T3 All serviced - valid=0
[PASS] T4 All spikes - cycle 0 - valid
[PASS] T4 All spikes - cycle 0 - addr
[PASS] T4 All spikes - cycle 1 - valid
[PASS] T4 All spikes - cycle 1 - addr
[PASS] T4 All spikes - cycle 2 - valid
[PASS] T4 All spikes - cycle 2 - addr
[PASS] T4 All spikes - cycle 3 - valid
[PASS] T4 All spikes - cycle 3 - addr
[PASS] T4 All serviced - valid=0
[PASS] T5 Mask persists - valid stays 0
[PASS] T6 step_rst - re-grants neuron 0 - valid
[PASS] T6 step_rst - re-grants neuron 0 - addr
[PASS] T7 No spikes - valid=0
[PASS] T7 Still no spikes - valid=0
[PASS] T8 Initial spike - neuron 0 - valid
[PASS] T8 Initial spike - neuron 0 - addr
[PASS] T8 After servicing - valid=0
[PASS] T8 Late arrival - neuron 2 - valid
[PASS] T8 Late arrival - neuron 2 - addr
[PASS] T8 Late arrival serviced - valid=0

==========================
  AER HANDLER TB
  PASS: 29  FAIL: 0
==========================
$finish called at time : 346 ns : File "Z:/Coursework/EE531/Final/tb_aer_handler.sv" Line 167
```

Fig. 14: AER Handler Unit Test Verification

```
run all
[PASS] T1 Reset - accum = 0 (got 0)
[PASS] T2 Single positive spike (got 10)
[PASS] T3 Negative spike - accum = -10 (got -10)
[PASS] T4a Spike addr 2 (got 40)
[PASS] T4b Spike addr 3 (got -60)
[PASS] T5 accum_rst clears to 0 (got 0)
[PASS] T6 No accumulation when valid=0 (got 0)
[PASS] T7 Positive saturation - clamped at MAX_VAL (got 32767)
[PASS] T8 Negative saturation - clamped at MIN_VAL (got -32768)
[PASS] T9 Back-to-back all addrs - accum = -60 (got -60)

==================================
  SYNAPSE ACCUMULATOR TB
  PASS: 10  FAIL: 0
==================================
$finish called at time : 22290 ns : File "Z:/Coursework/EE531/Final/tb_synapse_accumulator.sv" Line 165
```

Fig. 15: Synapse Accumulator Unit Test Verification

```
# run all
--RESET--
--WEIGHTS LOADED--
--- Decoding Finished ---
Neuron 0 LPF:  4329, Weight: 2.000000
Neuron 1 LPF:  4329, Weight: -1.000000
Neuron 3 LPF:  4329, Weight: 0.250000
Neuron 6 LPF:  4329, Weight: -0.500000
Neuron 10 LPF:  1923, Weight: 127.996094
Neuron 12 LPF:  2161, Weight: -128.000000
Neuron 15 LPF:  4329, Weight: 0.003906
Raw Accumulator: fff8dad59
Final Prediction (Hex): ff8dad59
Final Prediction (Decimal Fixed-Point): -114.322861
Match Found at Neuron Index: 0
```

Fig. 16: Output Decoder Unit Test Verification

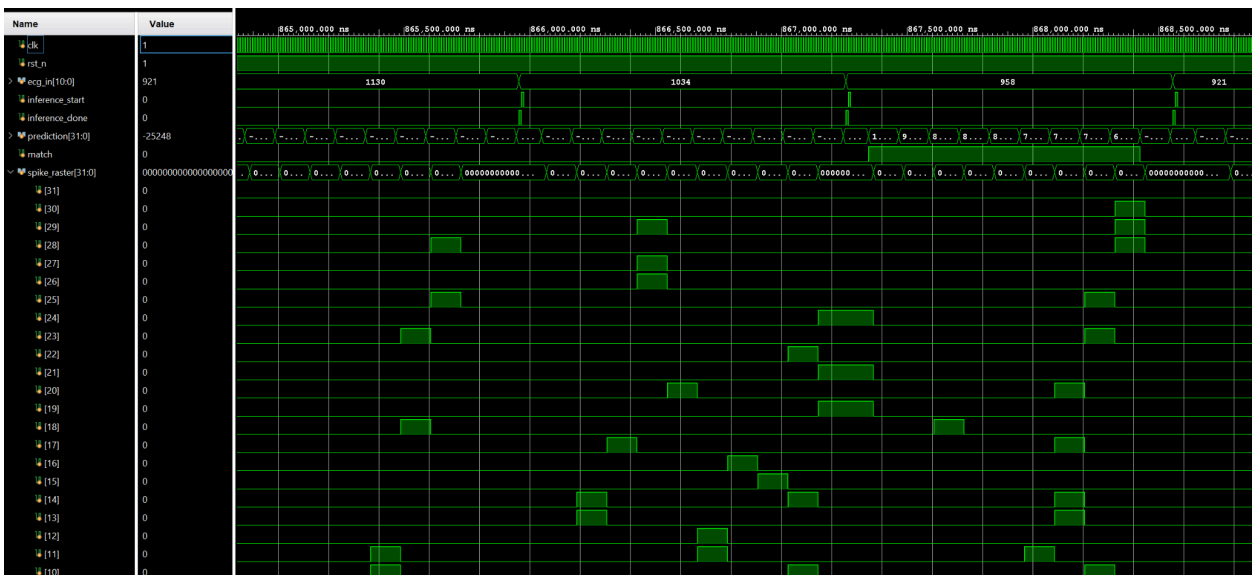

Fig. 17: Top Module Verification

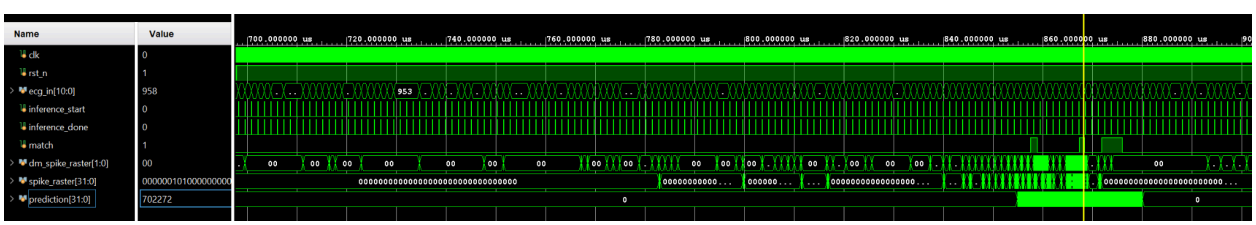

Fig. 18: Top Module Verification

Config.json

```
{
    "DESIGN_NAME": "snn_top",
    "VERILOG_FILES": [
        "dir::snn_top.sv",
        "dir::delta_mod.sv",
        "dir::aer_handler.sv",
        "dir::synapse_accumulator.sv",
        "dir::lif_neuron_fixed.sv",
        "dir::output_decoder.sv",
        "dir::lpf_unit.sv"
    ],
    "CLOCK_PERIOD": 25.0,
    "CLOCK_PORT": "clk",
    "CLOCK_NET": "clk",
    "pdk::sky130A": {
        "MAX_FANOUT_CONSTRAINT": 15,
        "FP_CORE_UTIL": 40,
        "PL_TARGET_DENSITY_pct": 45,
        "SYNTH_STRATEGY": "AREA 0",
        "PL_RESIZER_BUFFER_INPUT_PORTS": true,
        "PL_RESIZER_DESIGN_OPTIMIZATIONS": true,
        "PL_RESIZER_REPAIR_TIE_FANOUT": true,
        "DPL_CELL_PADDING": 2
    }
}
```

Modules, Testbenches, Netlist, etc.

Please see the zip folder uploaded alongside this report for the organized files. If we included everything in this document, it would be too long and difficult to find anything.